

Sensors Laboratory Experiment - 4

Human Activity Recognition Using Smartphone Accelerometer

Roll No. : ES24f1100064

Name : Arpit Katiyar

Objective :

To acquire accelerometer data from a smartphone and analyze it to detect human activities such as walking, climbing stairs, and descending stairs. The experiment aims to process the sensor data by calculating acceleration magnitude, applying filtering techniques, and implementing a peak detection algorithm to accurately count steps and classify different types of motion.

Theory :

An accelerometer measures both static acceleration (due to gravity) and dynamic acceleration (due to movement). The sensor records values along three axes: X, Y, and Z. Step Detection Algorithm: 1. Compute the magnitude of acceleration: ($A = \sqrt{A_x^2 + A_y^2 + A_z^2}$) 2. Apply a moving average filter to smoothen the signal and reduce noise. 3. Detects peaks in the filtered data using a peak detection algorithm. A suitable threshold is chosen to avoid false detections. Verification: To validate the detected steps, audio recordings of verbal step counts were also captured. Peaks from both accelerometer data and audio signals were compared on the same timeline.

Data Collection Summary :

The phone was held horizontally while walking. Both accelerometer data and audio step counts were recorded simultaneously. After 10 steps, the data was saved as a .csv file and audio as .m4a (later converted to .wav). Each dataset contained about 2000–2500 samples.

Description of Tools Used :

Phone model - Apple Iphone 12 Pro
Sampling rate selected - 100Hz
App selected - Physics Toolbox Sensor Suite
Method of data transfer - .csv (acceleration data)

Initial Testing of Accelerometer :

When placed flat, X and $Y \approx 0$, $Z \approx 0.98$
When vertical (width down), $Y \approx 0.98$
When vertical (length down), $X \approx 0.98$
In tilted positions, all three axes show varying values.
During movement, values continuously fluctuate.

Specifications of the accelerometer :

Based on available teardown analyses and technical documentation, the iPhone 12 Pro is equipped with a MEMS-based three-axis accelerometer as part of a Bosch BMI260 6-axis Inertial Measurement Unit (IMU), which also includes a three-axis gyroscope.

The table below outlines the detailed technical specifications for this accelerometer, providing a more technical look at its capabilities.

Specification	Value / Range
Model	Bosch Sensortec BMI260
Type	16-bit triaxial digital accelerometer
Acceleration Range	$\pm 2g$ / $\pm 4g$ / $\pm 8g$ / $\pm 16g$ (selectable scales)
Sensitivity	16,384 LSB/g (at $\pm 2g$) down to 2,048 LSB/g (at $\pm 16g$)
Resolution	16-bit / 0.06 mg/LSB
Output Data Rate (ODR)	Programmable, up to 1.6 kHz
Noise Density	160 $\mu g/\sqrt{Hz}$ (high performance mode)
Zero-g Offset	± 20 mg (typical)
Shock Survival	10,000 g for 0.2 ms
Operating Voltage	1.71 V to 3.6 V
Current Consumption	700 μA (full ODR), down to 30 μA (low-power modes)
Operating Temperature	-40°C to +85°C

Algorithm Description :

1. Preprocessing

The accelerometer data from X, Y, and Z axes is combined to compute the acceleration magnitude, making the signal independent of orientation. A moving average filter is applied to smooth the data and remove noise.

2. Peak Detection

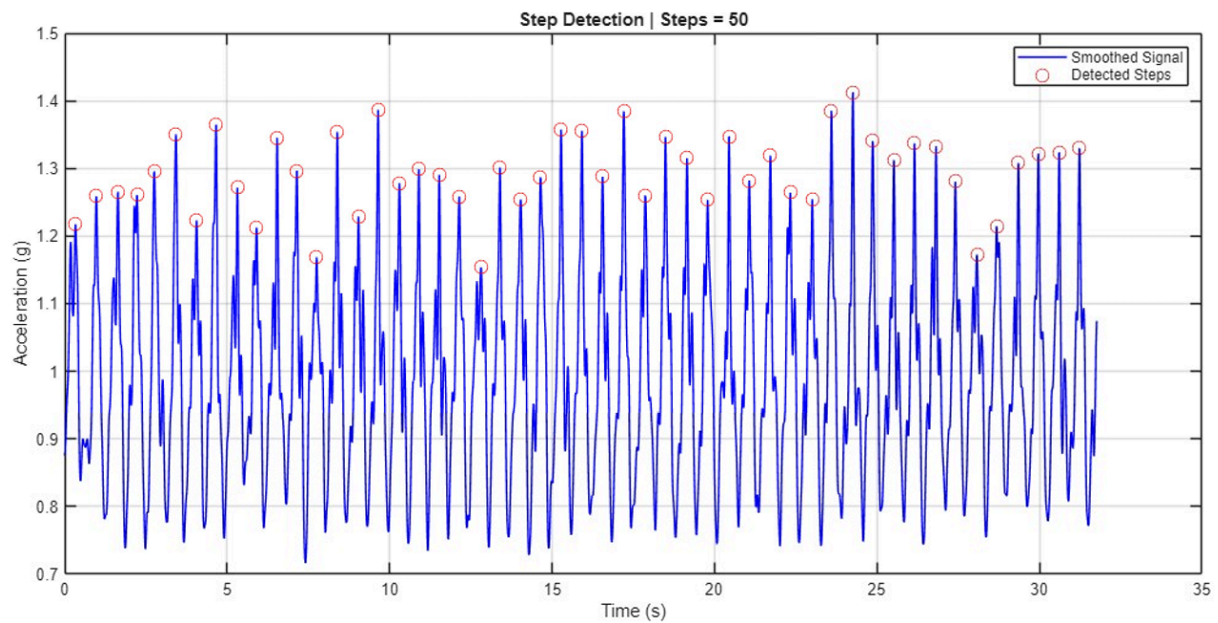
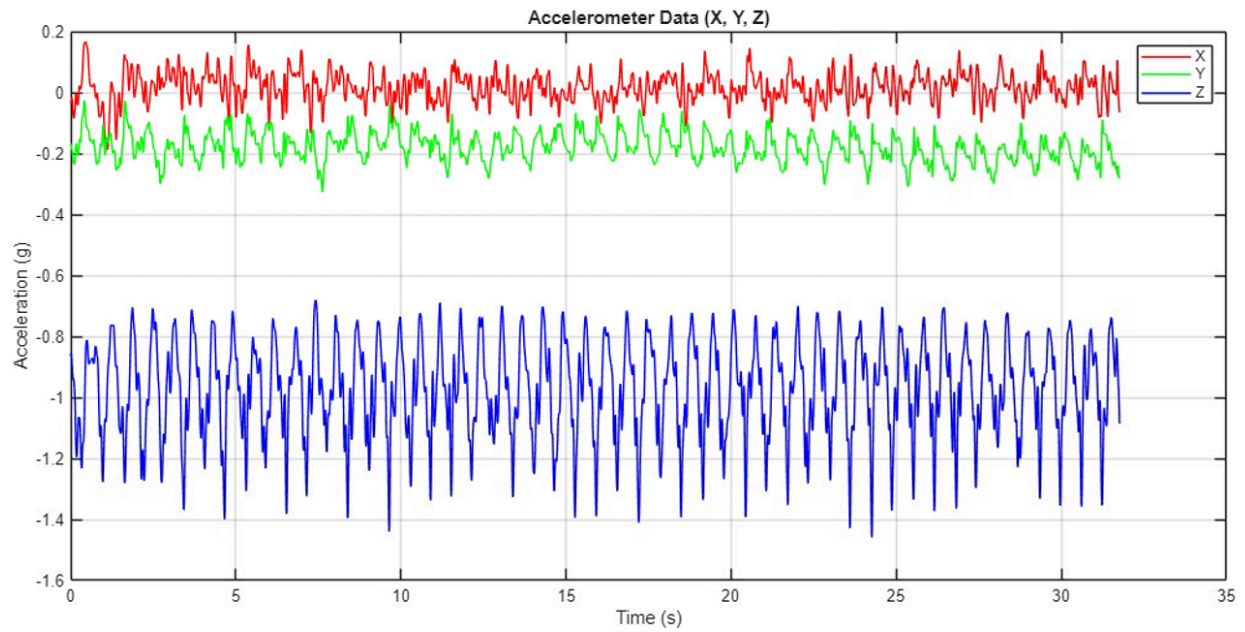
Peaks in the smoothed signal are detected using a threshold and minimum time interval. Each peak represents a step, and counting these peaks gives the total number of steps.

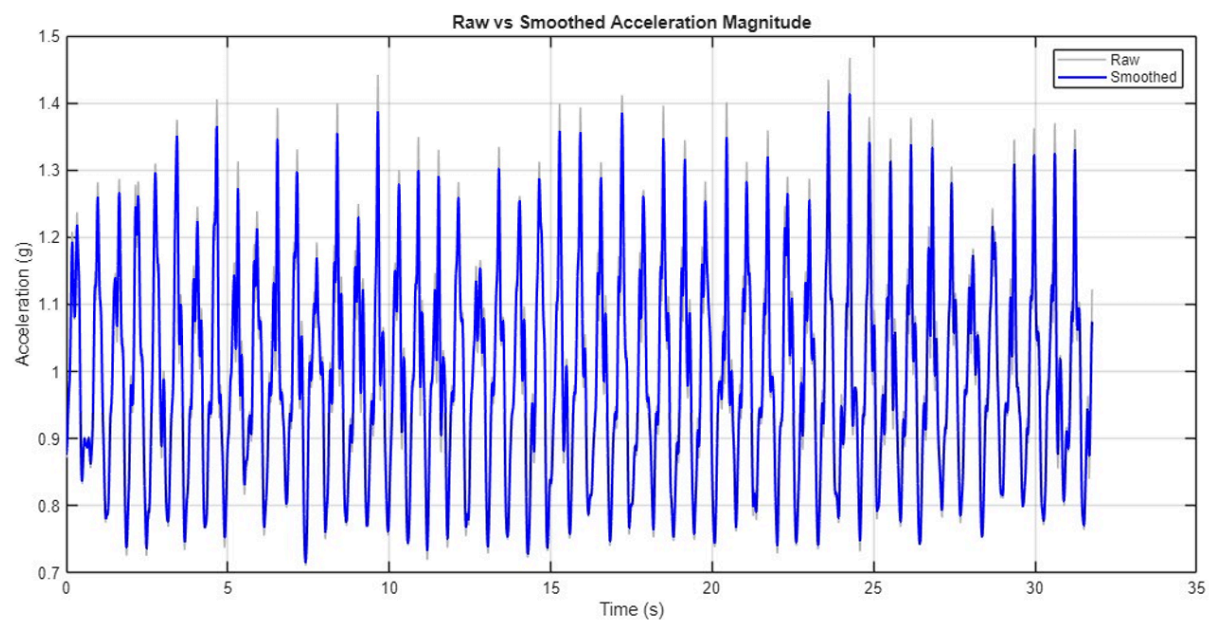
3. Activity Classification

The activity is identified based on the frequency and pattern of detected peaks. Walking, climbing, and descending stairs are differentiated using step frequency and signal characteristics.

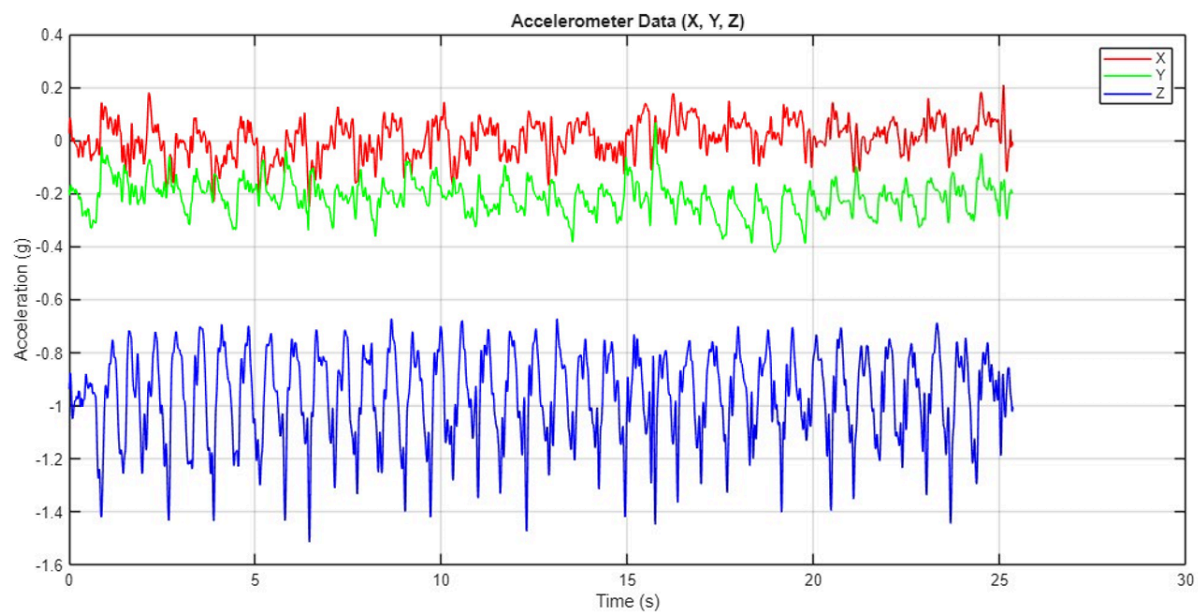
Results :

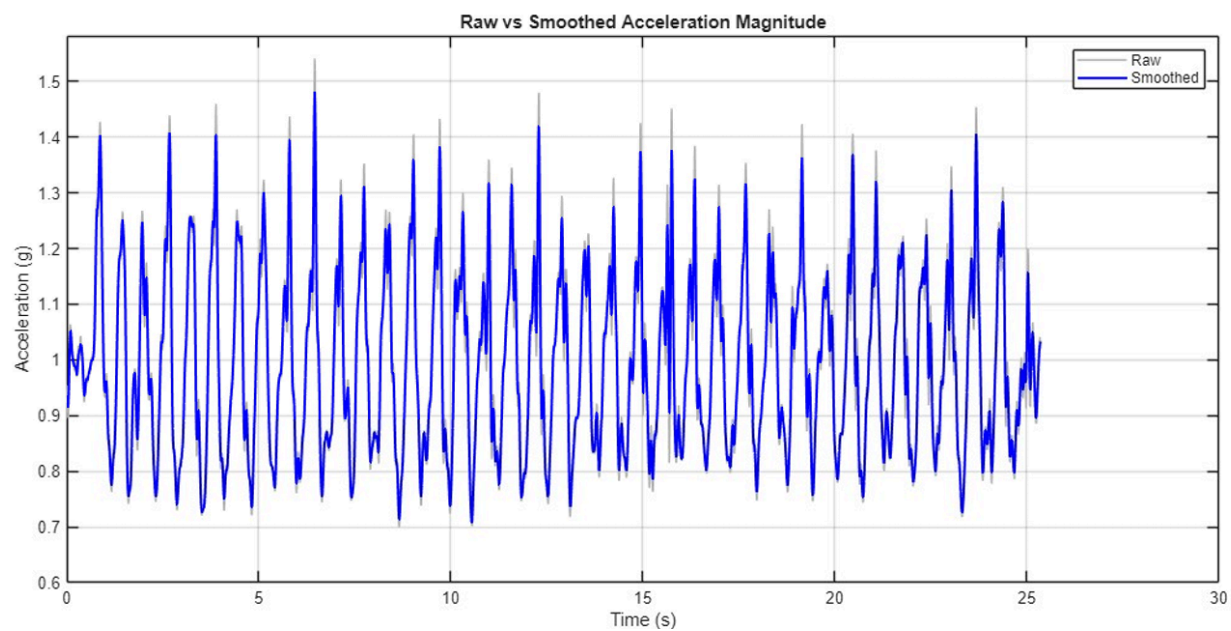
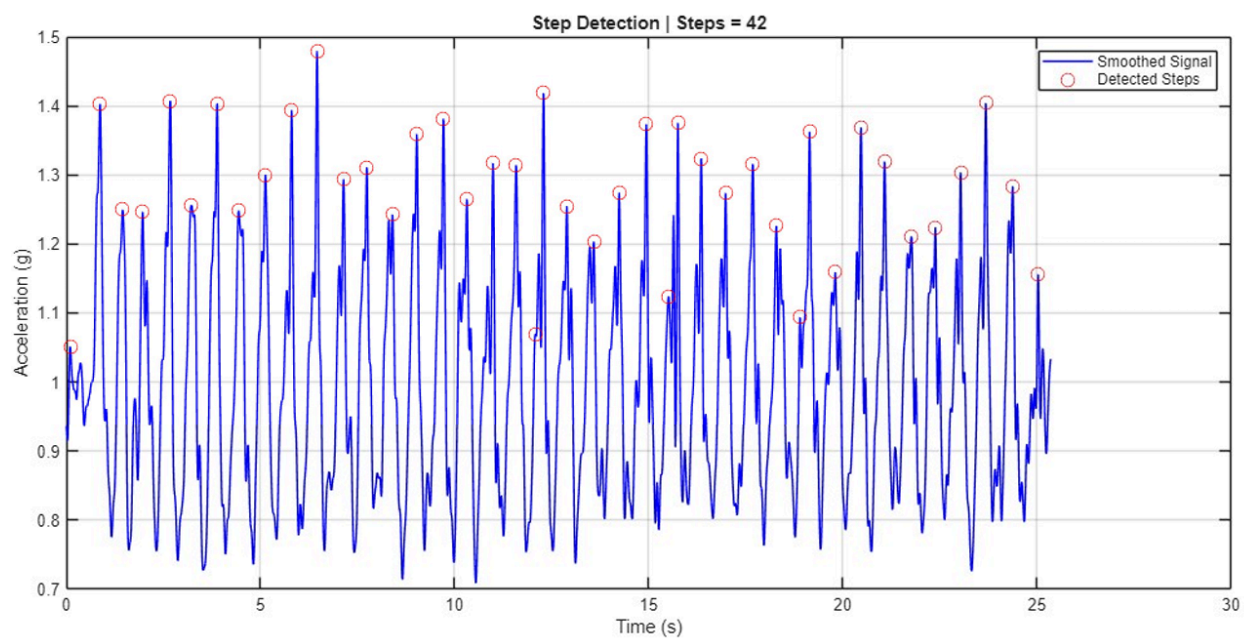
Walking on a Flat Surface :



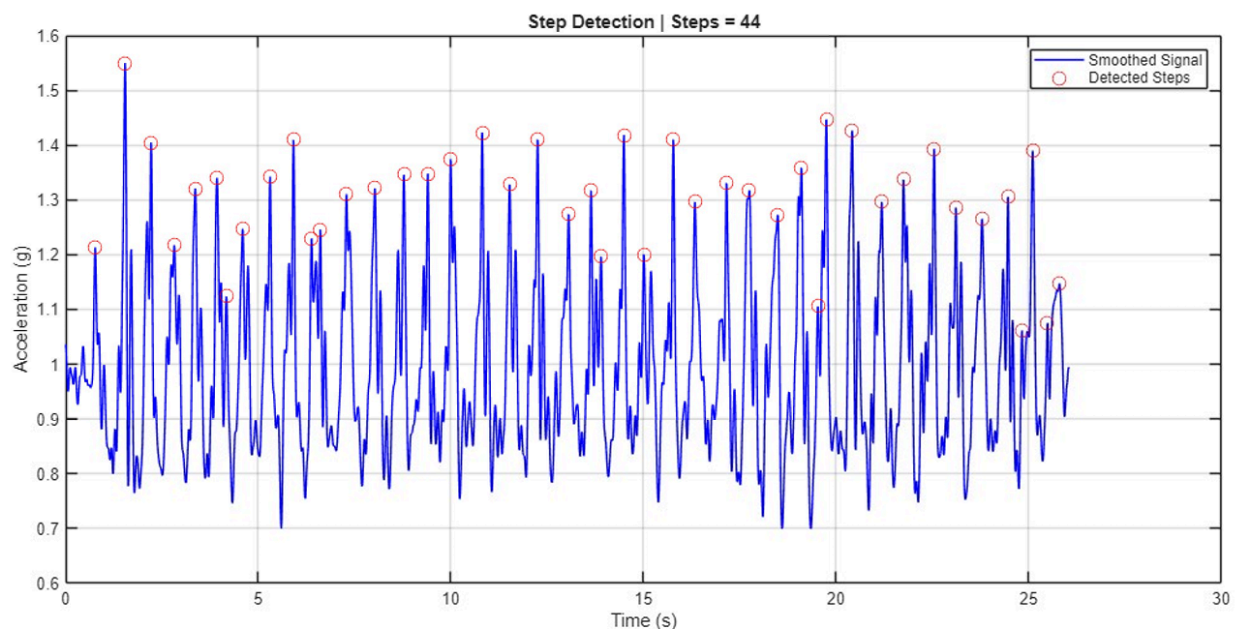
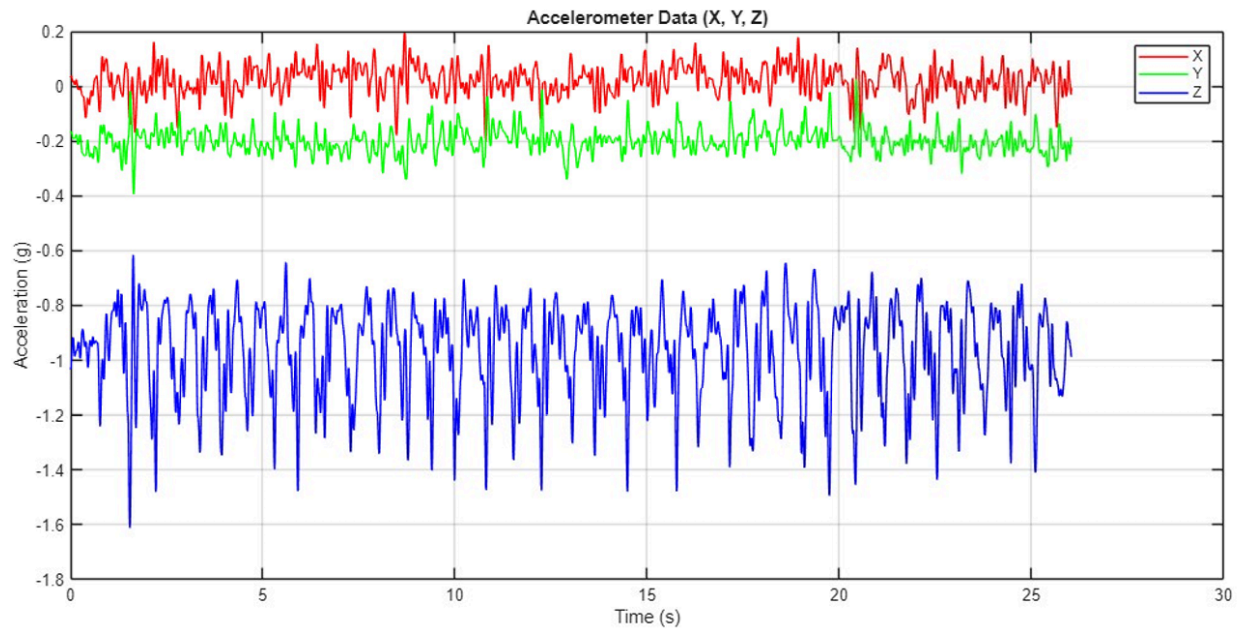


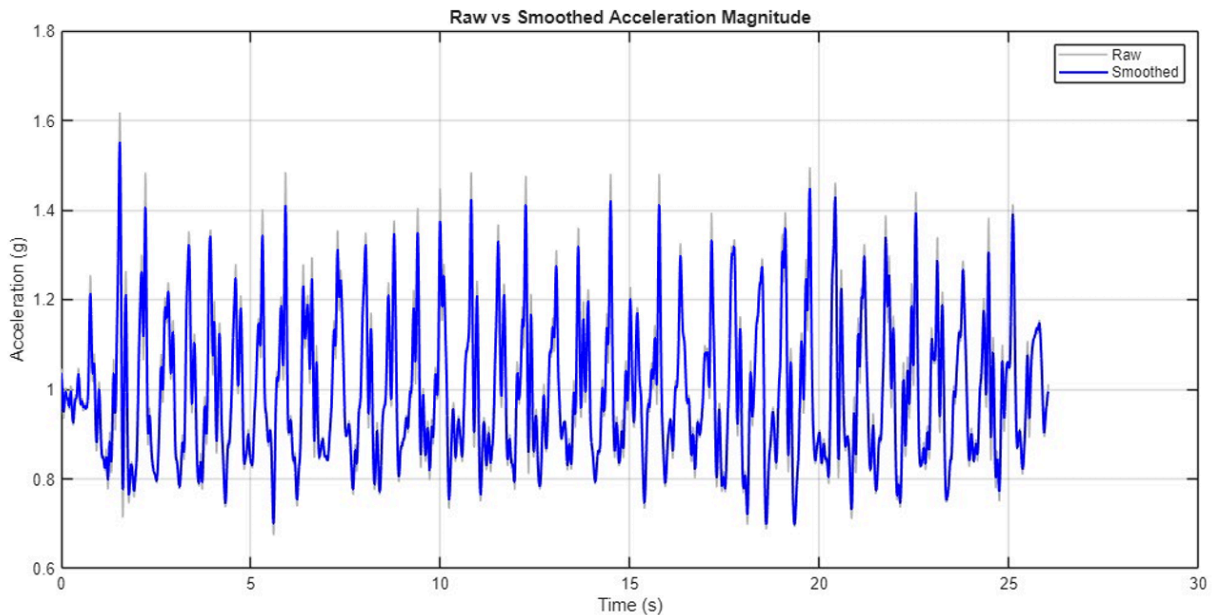
Climbing Up the Stairs :





Climbing Down the Stairs :





Discussion :

The algorithm reliably detects steps, with step peaks closely matching audio counts. Some redundant peaks occur, which can be minimized by refining the thresholding mechanism.

Threshold values were manually chosen by inspecting raw data; automation would improve efficiency.

Activity-specific patterns were observed:

- o Flat walking: mixed maxima and minima.
- o Downstairs: pronounced maxima.
- o Upstairs: distinct maxima and minima.

These distinctions can be used to classify different activities automatically.

Conclusion :

This experiment demonstrated the effective use of smartphone accelerometers in recognizing human activities. Distinct variations in acceleration patterns across activities provide a strong foundation for developing algorithms that can automatically classify movements such as walking, climbing, or descending stairs.

Appendix: Matlab Code :

The Matlab used for data processing and visualization are included below for reference. They are also saved as separate files in the project folder.

```

1  clc; clear; close all;
2
3  %% ----- STEP 1: IMPORT DATA -----
4  data = readtable('Walking_on_Floor.csv');
5
6  time = data.time;
7  x = data.x;
8  y = data.y;
9  z = data.z;
10
11 %% ----- STEP 2: COMPUTE MAGNITUDE -----
12 mag = sqrt(x.^2 + y.^2 + z.^2);
13
14 %% ----- STEP 3: SMOOTHING (MOVING AVERAGE) -----
15 windowSize = 5; % you can tune this
16 smoothMag = movmean(mag, windowSize);
17
18 %% ----- STEP 4: PLOT RAW vs SMOOTH -----
19 figure;
20 plot(time, mag, 'Color', [0.7 0.7 0.7]); hold on;
21 plot(time, smoothMag, 'b', 'LineWidth', 1.5);
22 title('Raw vs Smoothed Acceleration Magnitude');
23 xlabel('Time (s)');
24 ylabel('Acceleration (g)');
25 legend('Raw', 'Smoothed');
26 grid on;
27
28 %% ----- STEP 5: PEAK DETECTION (STEP COUNT) -----
29 % Tune these values based on your data
30 minPeakHeight = 1.05; % since your data is in g (~1 baseline)
31 minPeakDistance = 20; % depends on sampling rate
32
33 [peaks, locs] = findpeaks(smoothMag, ...
34     'MinPeakHeight', minPeakHeight, ...
35     'MinPeakDistance', minPeakDistance);
36
37 step_count = length(peaks);
38
39 %% ----- STEP 6: PLOT PEAKS -----
40 figure;
41 plot(time, smoothMag, 'b'); hold on;
42 plot(time(locs), peaks, 'ro', 'MarkerSize', 8);
43 title(['Step Detection | Steps = ', num2str(step_count)]);
44 xlabel('Time (s)');
45 ylabel('Acceleration (g)');
46 legend('Smoothed Signal', 'Detected Steps');
47 grid on;
48
49 %% ----- STEP 7: CADENCE (TIME BETWEEN STEPS) -----
50 time_diff = diff(time(locs)); % time between steps
51 avg_step_time = mean(time_diff);
52
53 step_frequency = 1 / avg_step_time; % steps per second
54
55 %% ----- STEP 8: ACTIVITY CLASSIFICATION -----
56 % You can tune these thresholds after observing your data
57
58 if step_frequency > 2
59     activity = 'Running';
60 elseif step_frequency > 1.5
61     activity = 'Climbing Stairs';

```

```

62 elseif step_frequency > 0.8
63     activity = 'Walking';
64 else
65     activity = 'Slow Movement / Standing';
66 end
67
68 %% ----- STEP 9: DISPLAY RESULTS -----
69 disp(['Step Count: ', num2str(step_count)]);
70 disp(['Step Frequency (steps/sec): ', num2str(step_frequency)]);
71 disp(['Detected Activity: ', activity]);
72
73 %% ----- OPTIONAL: SHOW ALL AXES -----
74 figure;
75 plot(time, x, 'r'); hold on;
76 plot(time, y, 'g');
77 plot(time, z, 'b');
78 title('Accelerometer Data (X, Y, Z)');
79 xlabel('Time (s)');
80 ylabel('Acceleration (g)');
81 legend('X', 'Y', 'Z');
82 grid on;

```